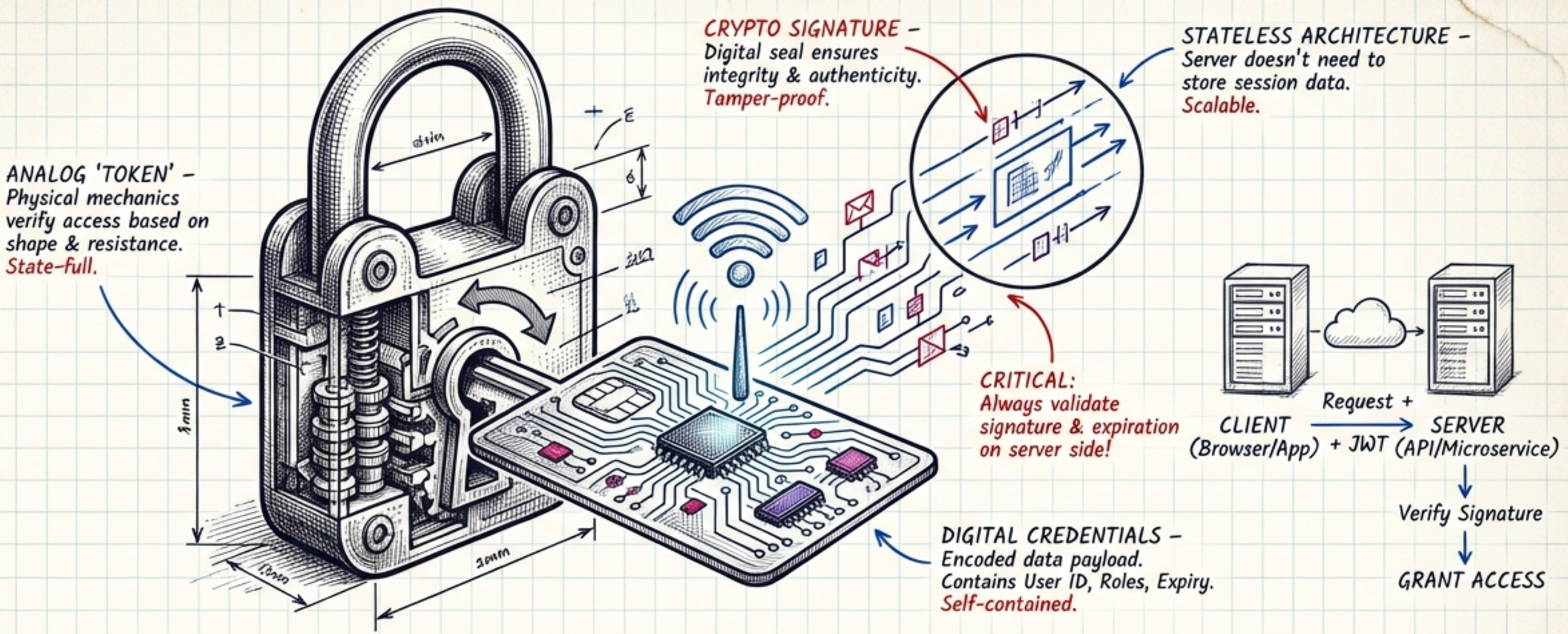


JWT EXPLAINED: THE FIELD GUIDE

Mechanics, Security, and Implementation Strategy

TAGS: AUTH PROTOCOLS / WEB STANDARDS / STATELESS ARCHITECTURE

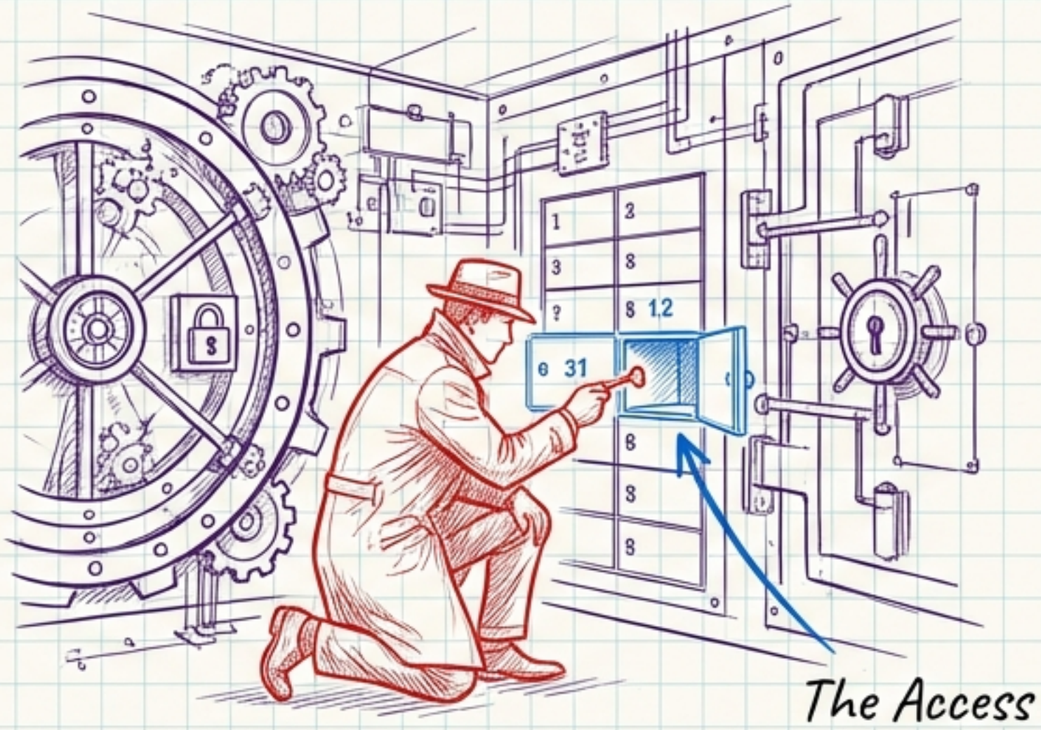
DOC ID: JWT-01

THE TWO PILLARS: AUTHENTICATION VS. AUTHORIZATION

WHO ARE YOU?

Authentication: Verifying identity via credentials (e.g., Email + Password).

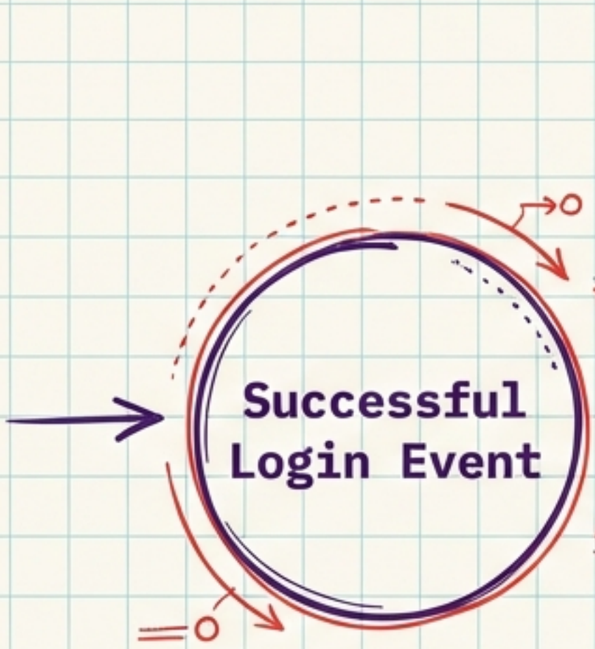
WHAT CAN YOU DO?



Authorization: Verifying permissions to access resources (e.g., viewing balance vs. transferring funds).

JWTs are the standard vehicle for the ***Authorization*** leg of this journey.

THE DELIVERABLES: LOGIN SUCCESS ARTIFACTS



The temporary pass.
Used in headers (Bearer Token).
Short-lived.

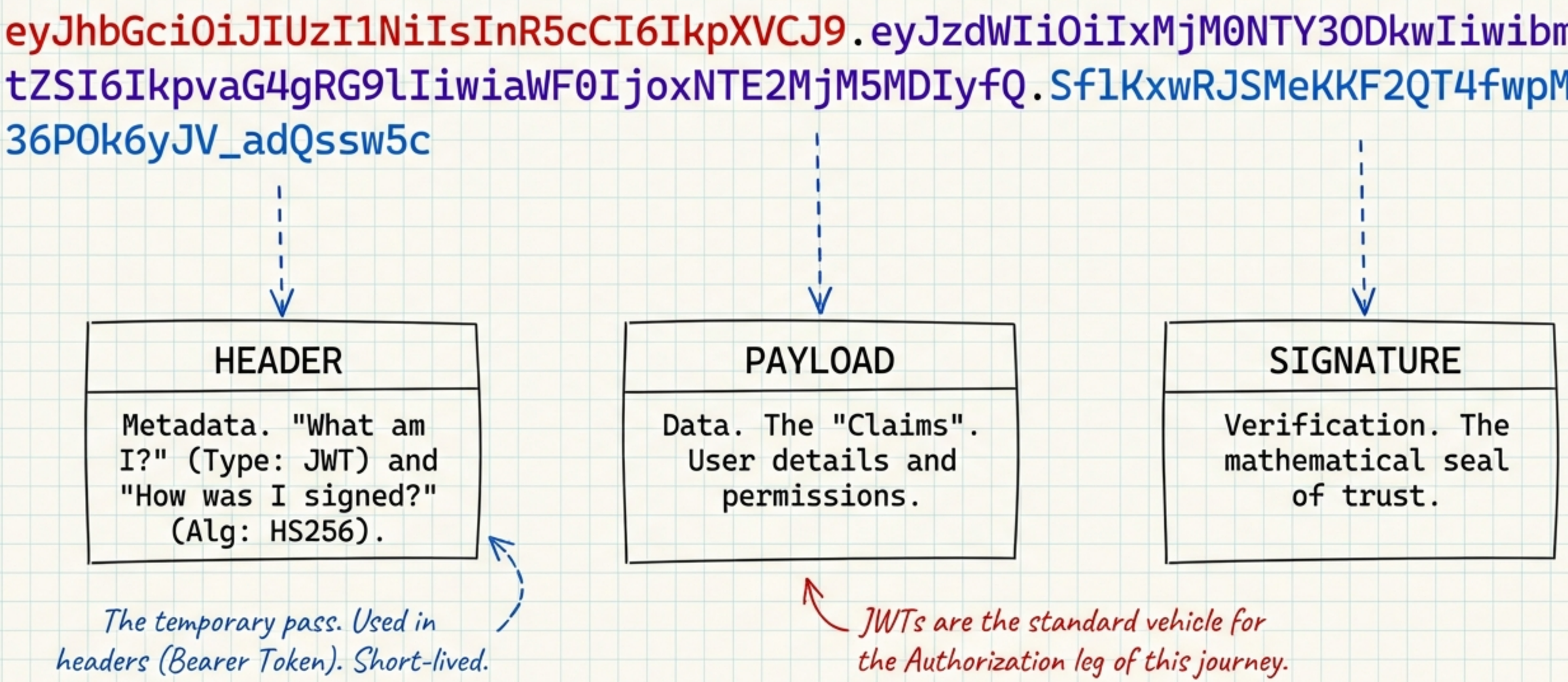
1. The Access Token (JWT)

The long-term ticket.
Stored securely (DB/Session)
to renew access.

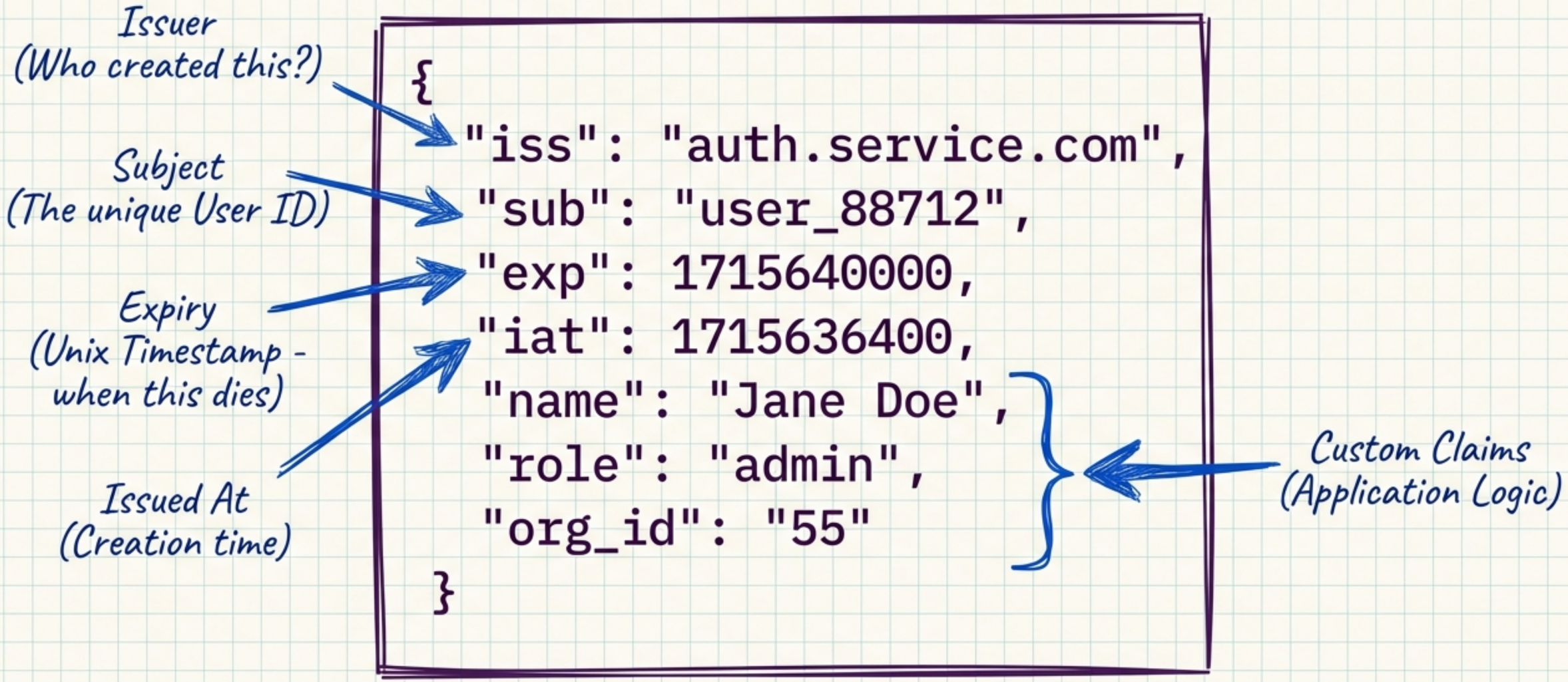
2. The Refresh Token

Upon authentication, the server issues these two critical keys.
This deck focuses first on the Access Token.

ANATOMY OF A TOKEN



INSIDE THE PAYLOAD: STANDARD & CUSTOM CLAIMS



THE SIGNATURE: THE MATHEMATICAL SEAL

The signature is a Hash generated from the content + a Server Secret.

```
HMACSHA256(
   encodedHeader + "." + encodedPayload,
   "very_secret_123"
)
```

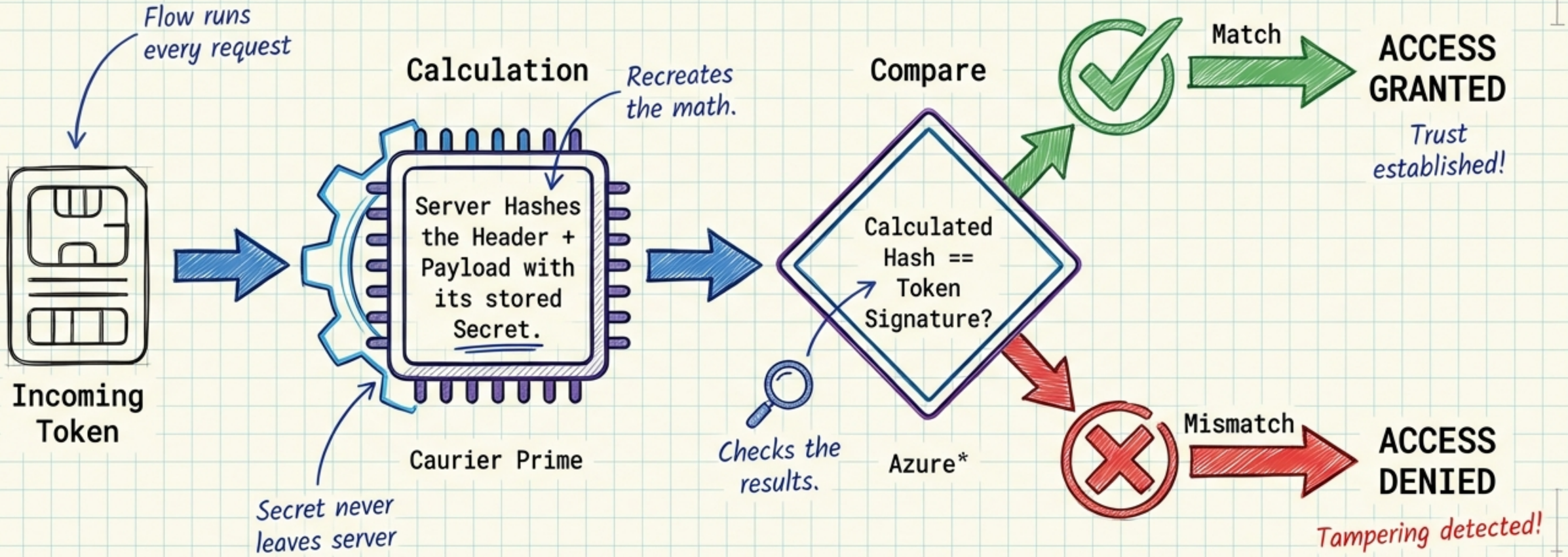
Translation: "I, the server, certify that the content in this Header and Payload is true because I signed it with my Secret."

The Signature

The mathematical seal of trust.

Key Concept: If any part of the header or payload changes, the resulting hash changes completely.

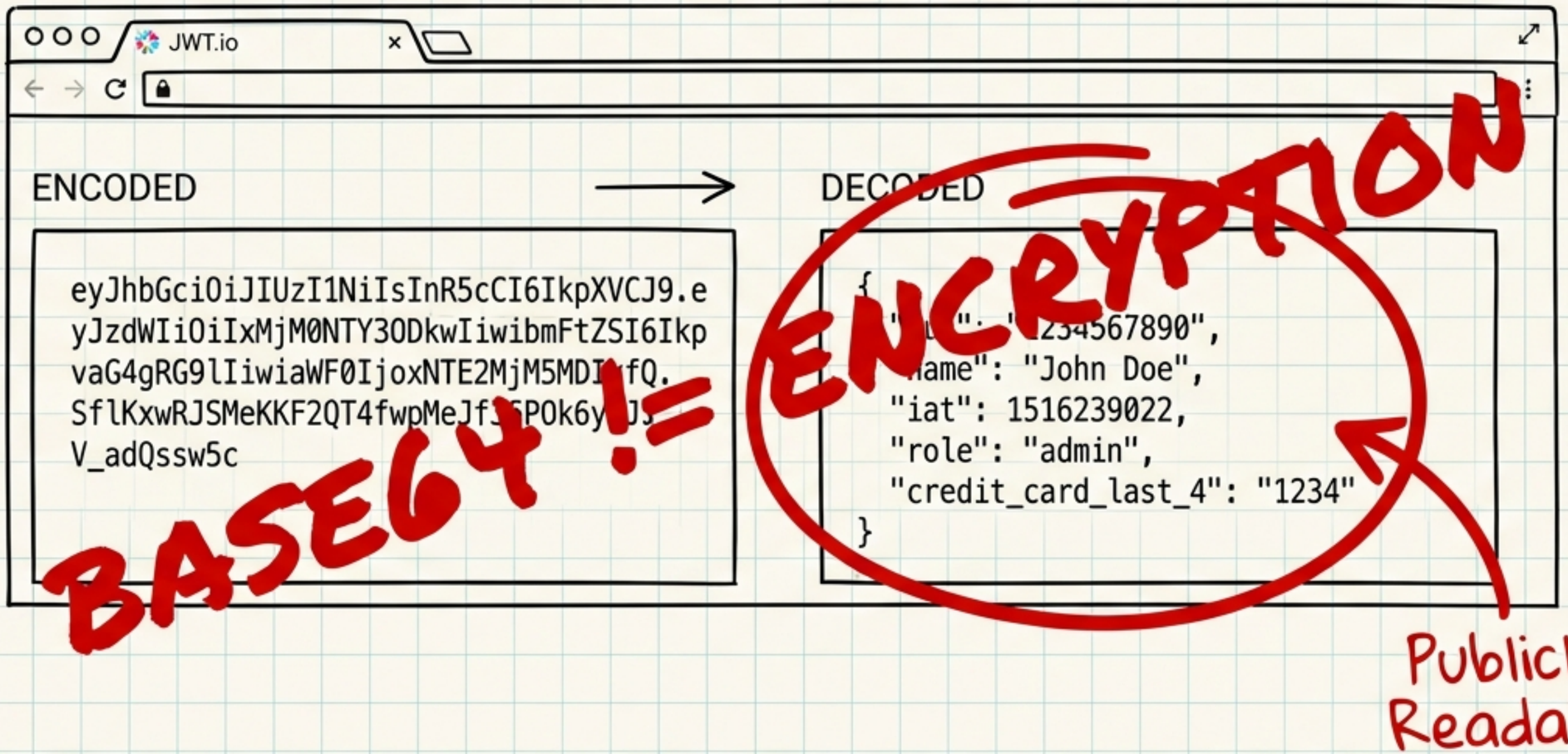
THE VERIFICATION FLOW



The server does not need to read the token to trust it; it simply recalculates the math.

The Engineering Field Journal

THE SECURITY ILLUSION



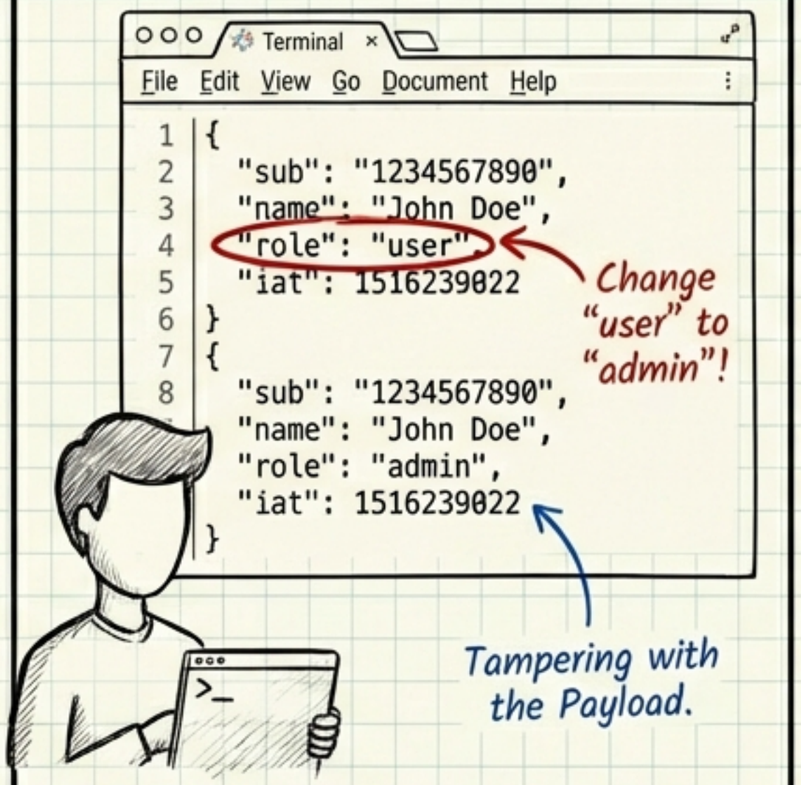
WARNING: Anyone who intercepts the token can read the payload.

The Rule: Never store sensitive data (passwords, PII, credit cards) in the JWT claims.

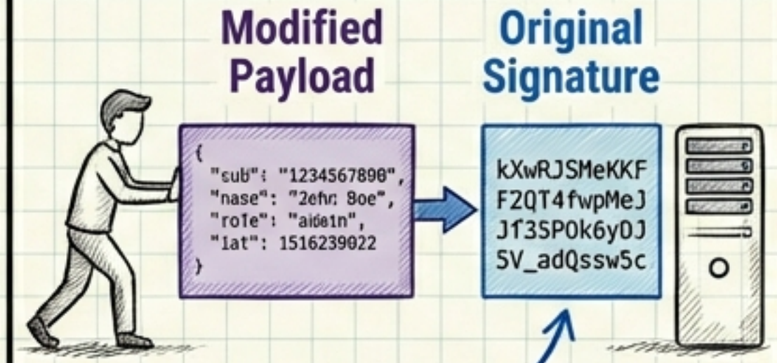
The Engineering Field Journal

THE HACK THAT FAILS

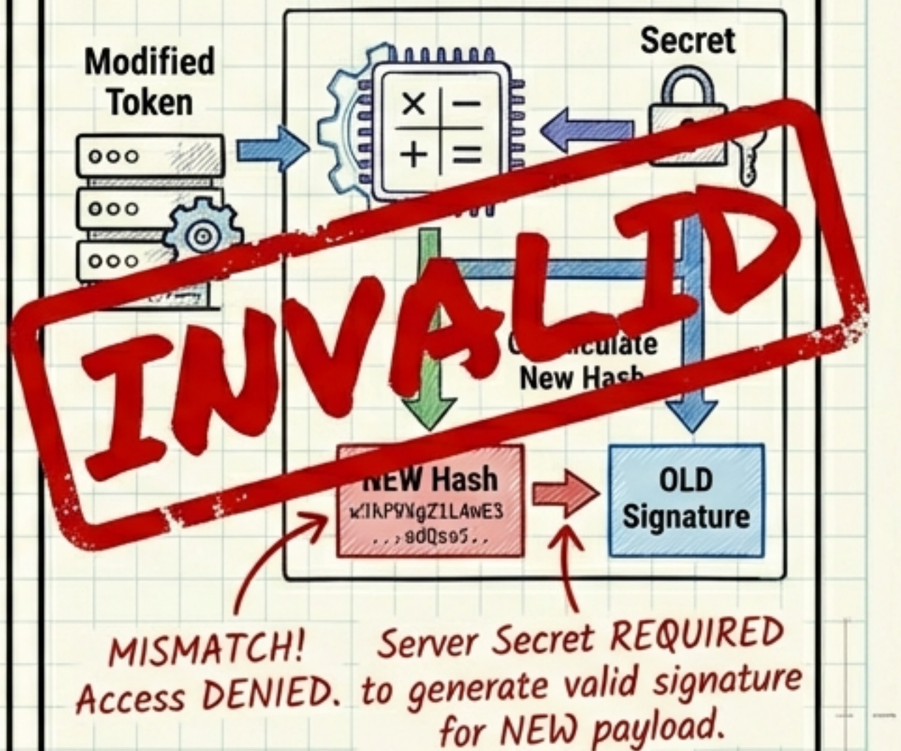
The Attempt



The Submission



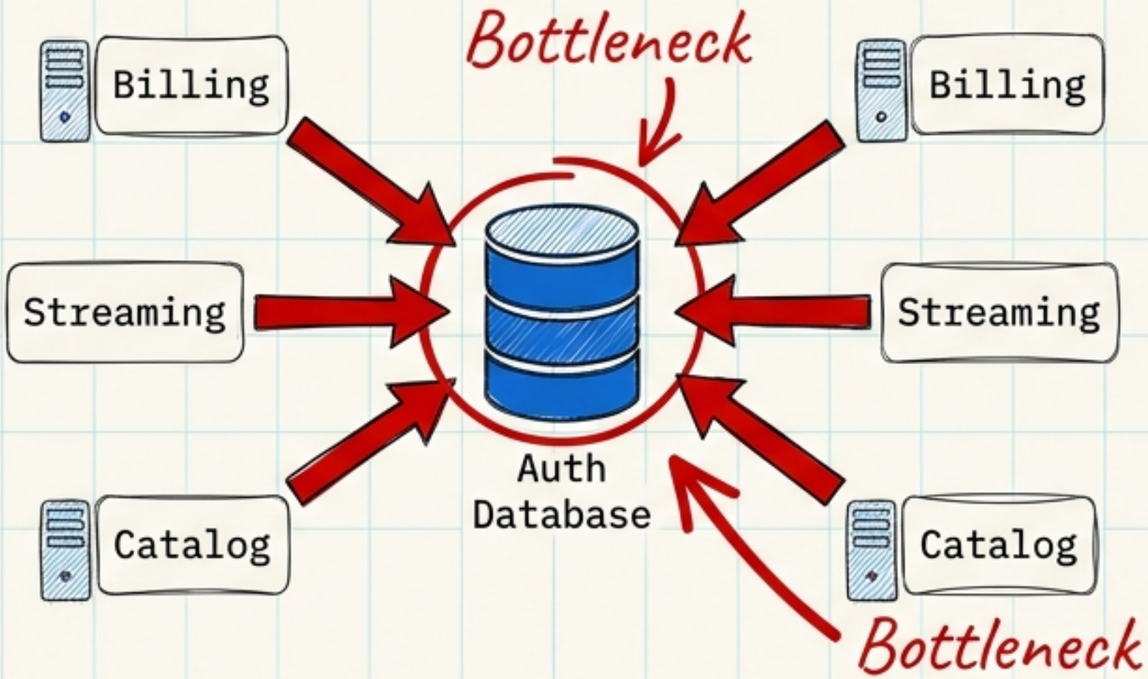
The Rejection



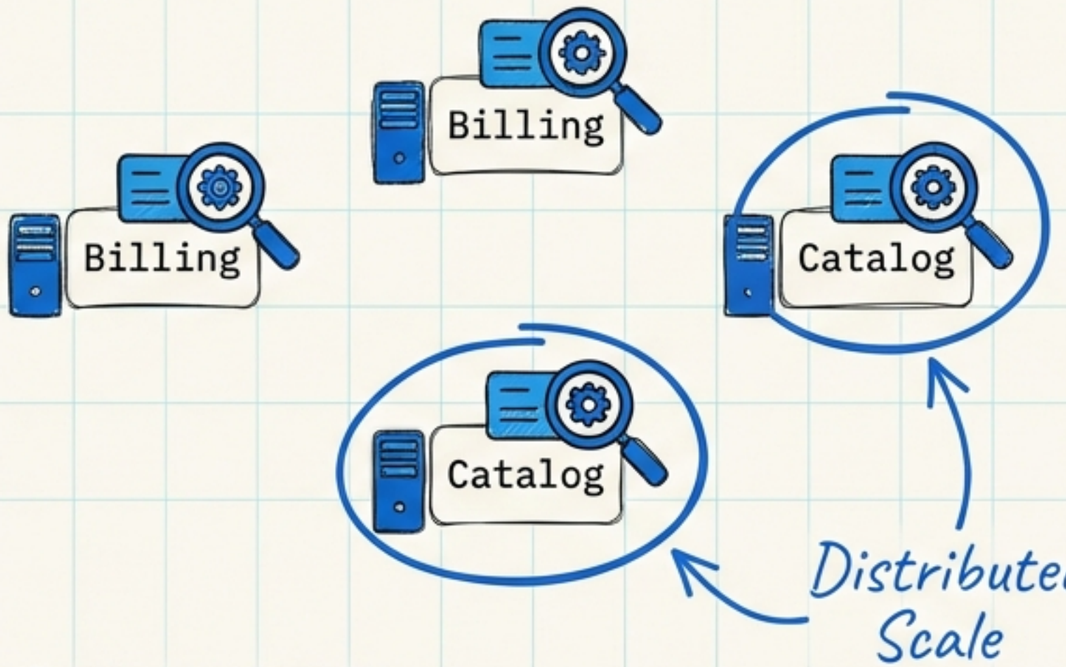
Without the Server Secret, you cannot generate a valid signature for your tampered payload.

THE SUPERPOWER: STATELESSNESS

The Old Way (Stateful)



The JWT Way (Stateless)



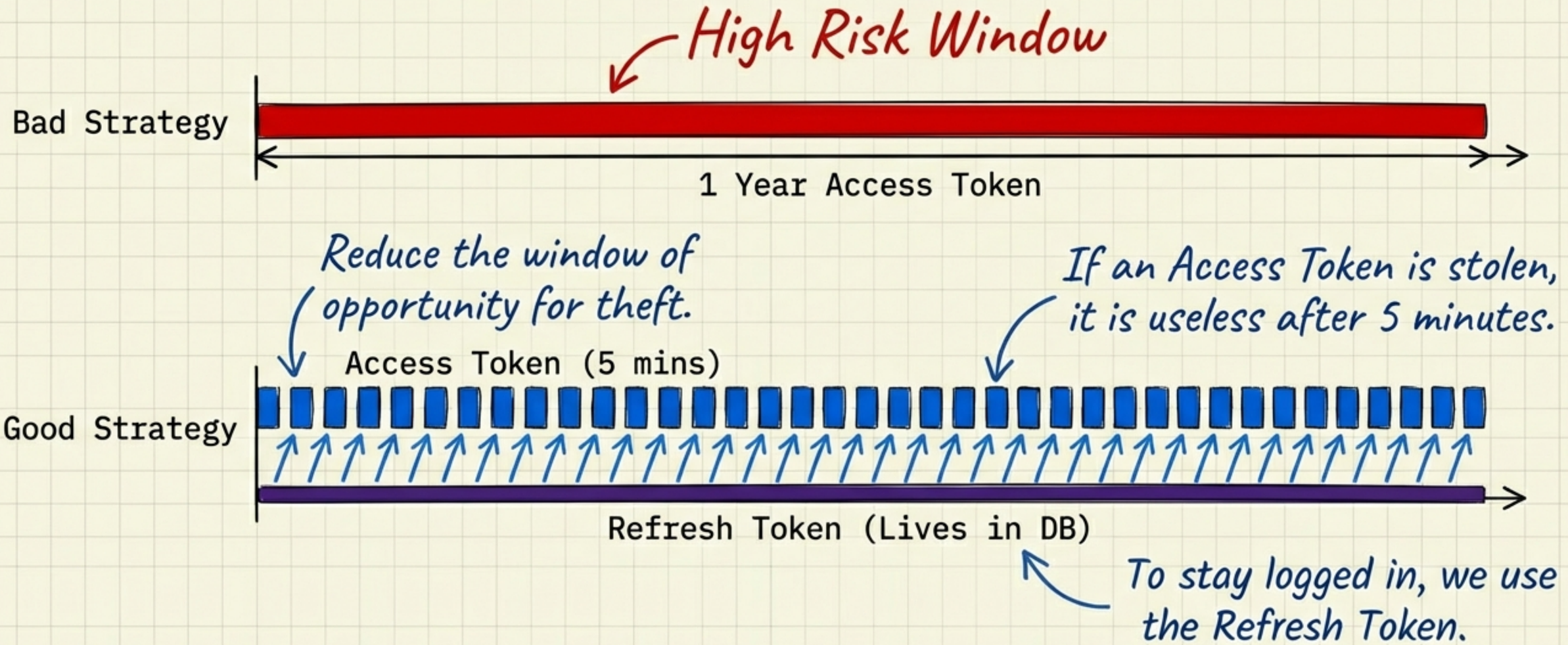
Context: Imagine Netflix. Thousands of microservices.
The token carries its own data (User ID, Org).
The services don't need to ask a central database "Who is this?"
for every single packet.
Result: Infinite horizontal scaling.

THE PROBLEM WITH LONGEVITY

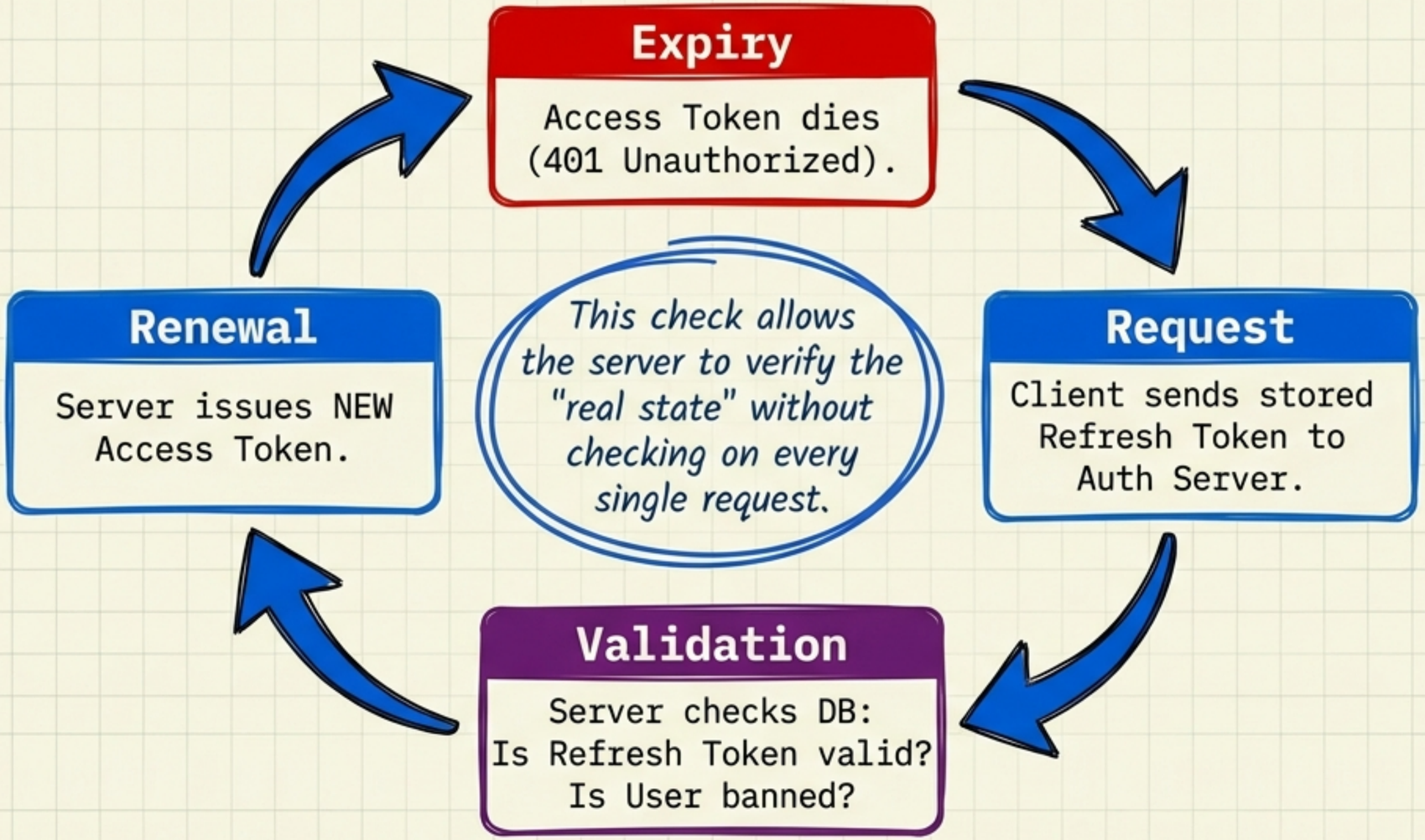
* **The Metaphor:** A JWT is like a passport. If stolen, the thief *becomes* you until it expires.
* **The Issue:** Since the server doesn't check the DB, it can't easily "ban" a specific token mid-flight. *Crucial flaw!*
* **The Risk:** If a user's role changes (Admin -> User), a long-lived token will still grant Admin rights until it expires. *Dangerous!* *Still Admin!*

Result: Unrevocable Access until Expiry.

STRATEGY: SHORT-LIVED ACCESS

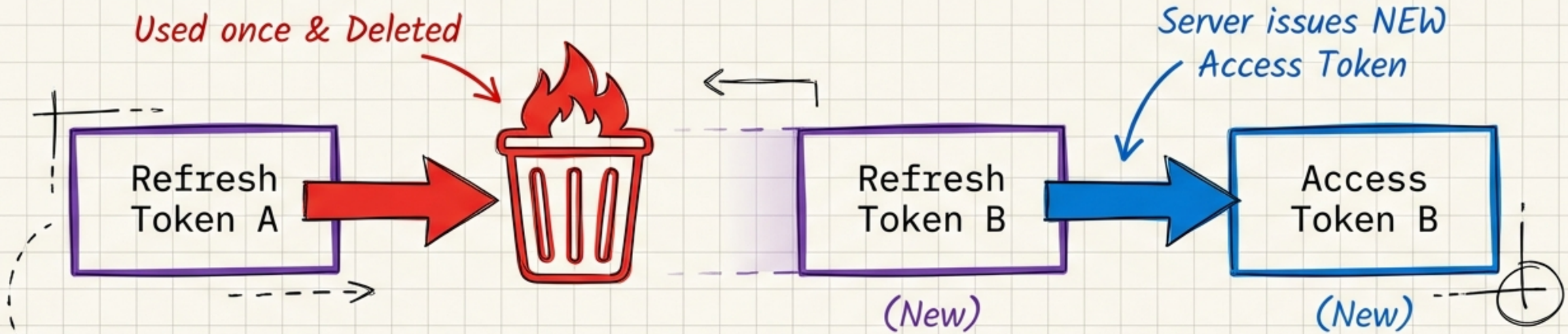


THE REFRESH FLOW



ADVANCED SECURITY: REFRESH TOKEN ROTATION

The Engineering Field Journal



Logic:

- Every time a Refresh Token is used, the server deletes the old one and issues a brand new one.

Benefit:

- Theft Detection. If an attacker tries to use an old Refresh Token (A), the server sees it has already been used and invalidates the entire chain.

ENGINEER'S RECAP

1. Stateless = Scalable. Use JWTs to validate across services without DB bottlenecks.
2. Untrusted Payload. Decoded != Decrypted. Untrusted Payload. Decoded != Decrypted. Never put secrets in the payload.
3. Short Lifespans. Access tokens should expire in minutes.
4. Rotation. Rotate refresh tokens on every use to prevent replay attacks.

Verify signatures. Trust the math.